Artificial Intelligence & Machine Learning – Task 3

Model Validation, Overfitting Control & Hyperparameter Tuning

Submitted by:

Namratha T G

1. Introduction

Machine learning models must not only perform well on training data but also generalize effectively to unseen data. One of the most common problems in machine learning is **overfitting**, where a model learns the training data too well and fails to perform accurately on new data.

The objective of this project is to build a house price prediction model using the **California Housing Dataset** and improve its reliability using **cross-validation** and **hyperparameter tuning**. Several regression models are evaluated and compared to determine the most suitable model for predicting house prices.

2. Dataset Description

The dataset used in this project is the **California Housing Dataset**, which contains housing information collected from the 1990 U.S. Census.

The dataset includes the following features:

- Median Income (MedInc)
- House Age (HouseAge)
- Average Rooms (AveRooms)
- Average Bedrooms (AveBedrms)
- Population
- Average Occupancy (AveOccup)
- Latitude
- Longitude

The target variable is **HousePrice**, which represents the median house value in California districts.

AI_ML_Task3_Model_Validation_Tuning.ipynb

Edit View Insert Runtime Tools Help

+ Code + Text Run all

```
data = fetch_california_housing(as_frame=True)

df = pd.concat([data.data, data.target.rename("HousePrice")], axis=1)

df.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	HousePrice
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

Figure 1: Sample of the California Housing Dataset

3. Data Preprocessing

Before training the machine learning models, several preprocessing steps were performed.

First, the dataset was loaded using the Scikit-learn library. The input features were separated from the target variable (HousePrice). After that, **feature scaling** was applied using the StandardScaler method. Feature scaling ensures that all input features are normalized and helps machine learning algorithms learn more efficiently.

Next, the dataset was divided into training and testing sets using an **80/20 split**. The training data was used to train the models, while the testing data was used to evaluate model performance.

4. Overfitting Detection

A **Decision Tree Regression** model was initially trained to analyze the difference between training performance and testing performance. The results showed that the training error was extremely small, while the

test error was significantly higher. This indicates that the model was memorizing the training data instead of learning general patterns.

This phenomenon is known as **overfitting**, where a model becomes too complex and performs poorly on unseen data.

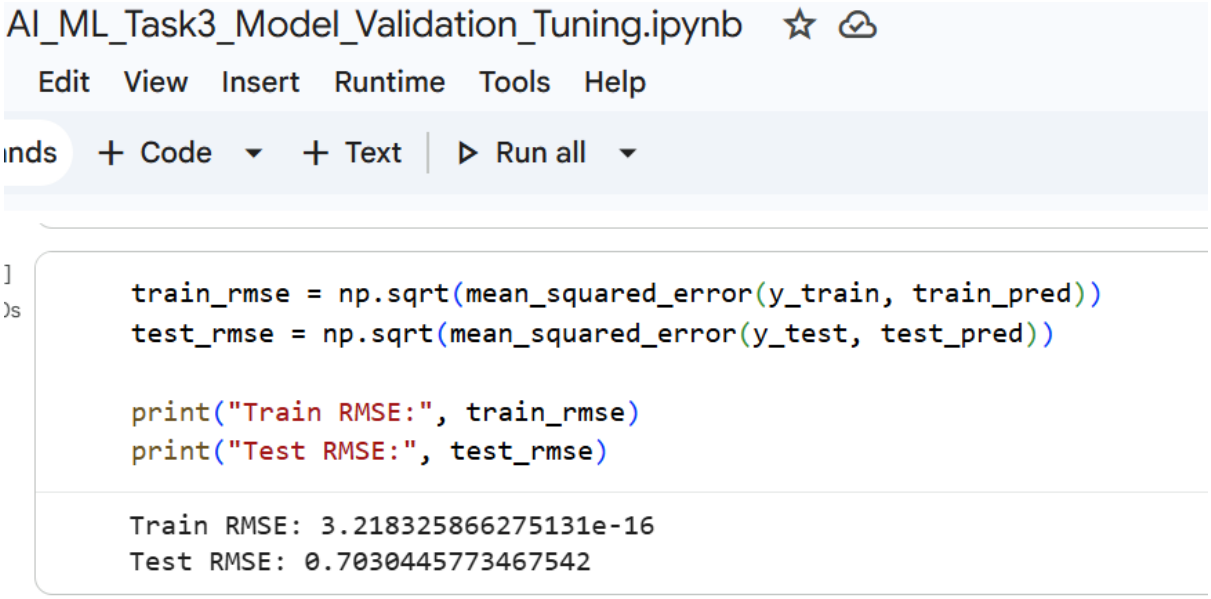

Figure 2: Overfitting Detection

5. Cross-Validation

To obtain a more reliable estimate of model performance, **5-fold cross-validation** was used.

Cross-validation splits the dataset into multiple subsets and evaluates the model several times using different training and testing combinations. The average performance across all folds provides a more stable and accurate performance estimate.

This technique reduces dependence on a single train-test split and ensures that the model performs consistently across different subsets of the data.

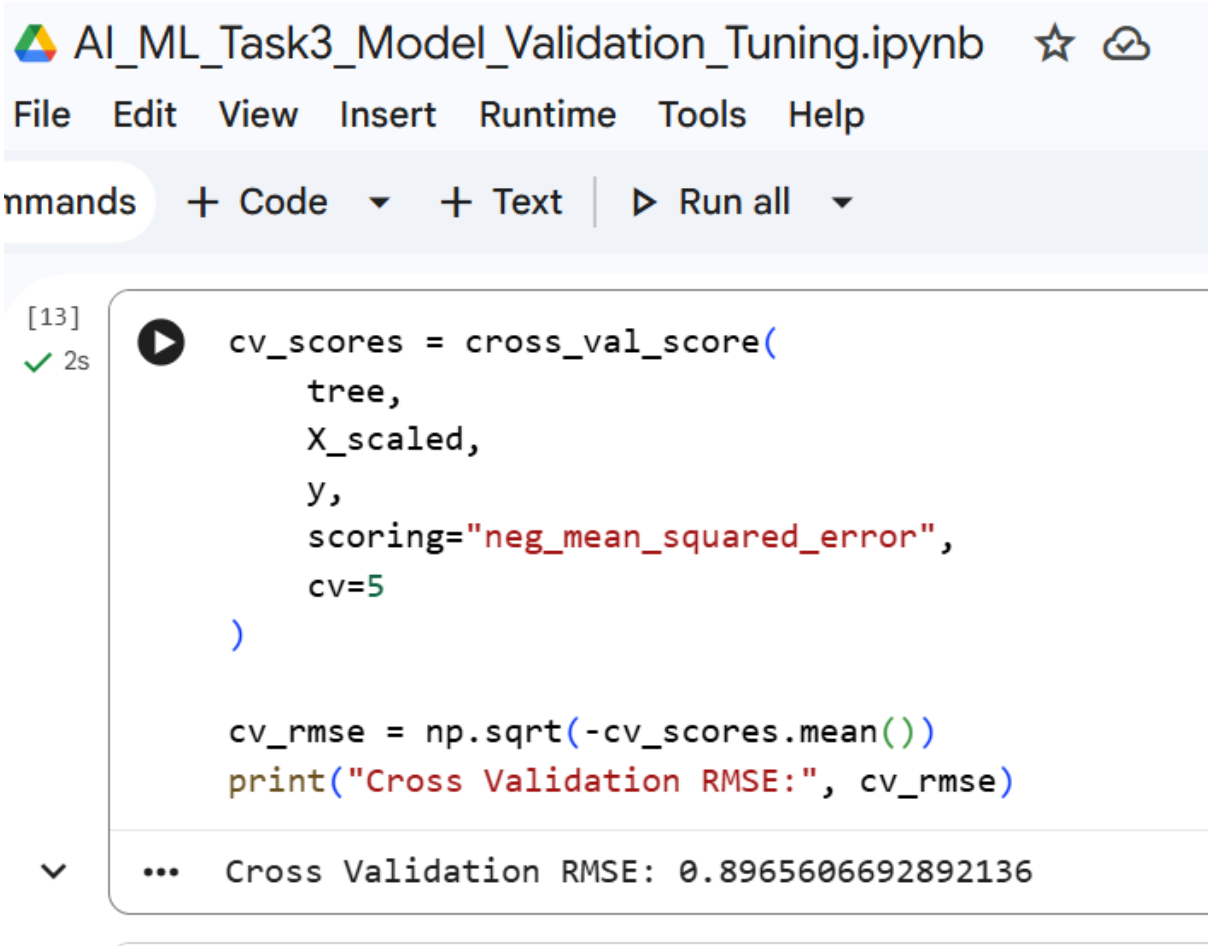

Figure 3: Cross-Validation Result

6. Hyperparameter Tuning

To reduce overfitting and improve model performance, **hyperparameter tuning** was performed using **GridSearchCV**.

Two important Decision Tree parameters were optimized:

- **max_depth:** controls the maximum depth of the decision tree
- **min_samples_split:** determines the minimum number of samples required to split a node

GridSearchCV tested several combinations of these parameters and selected the best configuration based on cross-validation performance.

The optimal parameters obtained were:

- max_depth = 10

- min_samples_split = 10

These parameters helped reduce model complexity and improve generalization.

7. Model Evaluation

The optimized Decision Tree model was evaluated using two important regression metrics:

RMSE (Root Mean Squared Error)
Measures the prediction error of the model. Lower values indicate better performance.

R^2 Score (Coefficient of Determination)
Represents how well the model explains the variation in the target variable.

The optimized model achieved:

- **RMSE ≈ 0.645**
- **R^2 ≈ 0.682**

These results show that the model can explain approximately **68% of the variation in house prices**.

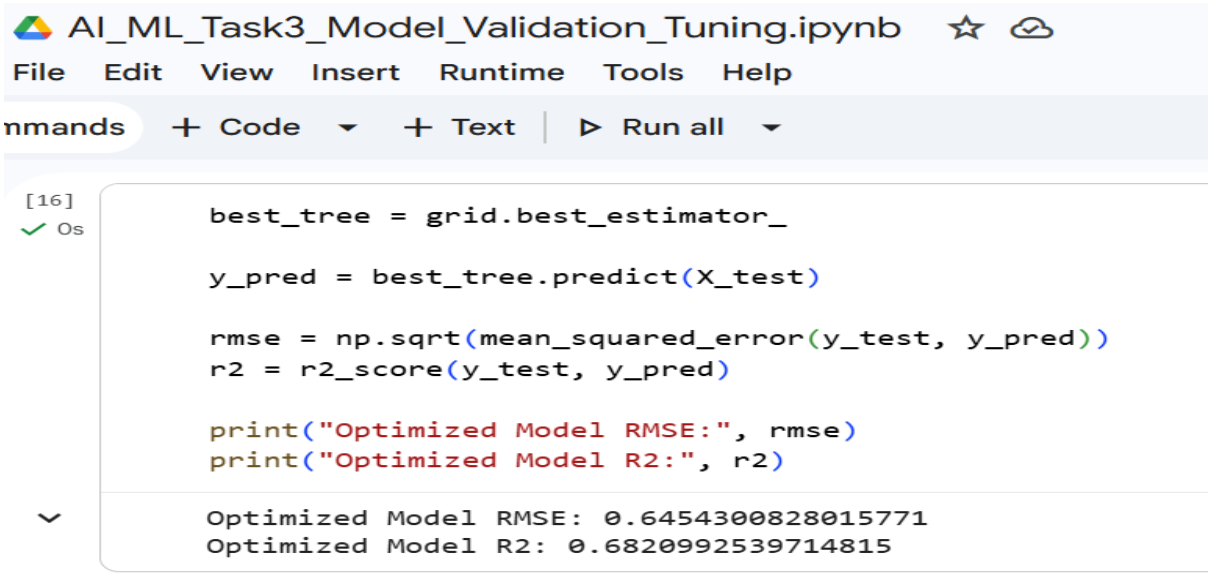

Figure 4: Optimized Model Performance

8. Model Comparison

Three regression models were evaluated and compared:

- Linear Regression
- Ridge Regression
- Tuned Decision Tree Regression

The models were compared based on RMSE and R^2 score.

AI_ML_Task3_Model_Validation_Tuning.ipynb

File Edit View Insert Runtime Tools Help

nmands + Code + Text Run all

[19]
0s

```
results = {
    "Model": ["Linear Regression", "Ridge Regression",
    "RMSE": [lr_rmse, ridge_rmse, rmse],
    "R2 Score": [lr_r2, ridge_r2, r2]
}

pd.DataFrame(results)
```

	Model	RMSE	R2 Score
0	Linear Regression	0.745581	0.575788
1	Ridge Regression	0.745554	0.575819
2	Tuned Decision Tree	0.645430	0.682099

Figure 5: Model Comparison Table

The results indicate that the **tuned Decision Tree model achieved the lowest RMSE and highest R^2 score**, making it the most suitable model for this prediction task.

9. Conclusion

In this project, overfitting was successfully identified and addressed using cross-validation and hyperparameter tuning techniques.

The Decision Tree model initially suffered from severe overfitting, but tuning its parameters significantly improved its generalization ability. Cross-validation ensured that the model evaluation was reliable, while GridSearchCV helped identify the best model configuration.

The final tuned Decision Tree model achieved the best balance between prediction accuracy and generalization performance. This demonstrates the importance of proper model validation and hyperparameter optimization in building reliable machine learning systems.

10. Final Deliverables

The final submission includes:

1. Jupyter Notebook containing the full implementation
2. Cross-validation results
3. Hyperparameter tuning results
4. Model comparison table
5. This 2–3 page project report